Stored Procedures, Transaction and cursor, Trigger

♣ Stored Procedure:
- Use of stored procedure, structure of stored procedure,
- Supported SQL statements in Procedures, creating dynamic procedure,
- Adding record to the table using procedure,
- Procedure with IN, OUT, INOUT parameter,
- Dropping procedure

♣Triggers: use, example

♣Transaction:
- MySQL transactions,
- Open and closing transaction, commit, rollback, savepoint in transaction,
- Table lock

♣Cursor:
- Use of cursor,
- Types of cursor,
- Opening a cursor, fetching a record from the cursor, cursor fetch statement, closing cursor

♣MySQL import & export:
- Import CSV File into MySQL Table,
- Export MySQL Table to CSV

Declaring variables

A variable is a named data object whose value can change during the stored procedure execution. You typically use variables in stored procedures to hold immediate results. These variables are local to the stored procedure. To declare a variable inside a stored procedure, you use the DECLARE statement.

```
Syntax: declare variable_name  datatype(size) [default default_value];
Example: declare x, y int default 0;
```

Assigning variables

Once a variable is declared, it is ready to use. To assign a variable a value, you use the SET statement:

```
Syntax:  set variable_name = value;
Example:  declare sum int default 0;
                set sum = 10;
```

- **Delimiter**: By default semicolon (;) delimiter is used to complete an SQL statement. But this causes problem in stored procedure because a procedure can have many statements and everyone must end with a semicolon. We have used double slash (//) instead of semicolon (;).

```
mysql> delimiter // ;
```

- **Check the privileges of the current user:**

CREATE PROCEDURE and CREATE FUNCTION require the CREATE ROUTINE privilege. They might also require the SUPER privilege, depending on the DEFINER value.

```
mysql> show privileges;
```

- **Local variables**

Local variables are declared within stored procedures and are only valid within the BEGIN…END block where they are declared. Local variables can have any SQL data type.

```
delimiter //
create procedure my_procedure_local_variables()
begin   /* declare local variables */
declare a int default 10;
declare b, c int;    /* using the local variables */
set a = a + 100;
set b = 2;
set c = a + b;
```

```
begin      /* local variable in nested block */
declare c int;
set c = 5;  /* local variable c takes precedence over the one of the same name declared in the enclosing block. */
select a, b, c;
end;
select a, b, c;
end//
```

Execute the procedure: call my_procedure_local_variables();

- **User variables**

In MySQL stored procedures, user variables are referenced with an ampersand (@) prefixed to the user variable name. Example: @x and @y

```
delimiter //
create procedure my_procedure_user_variables()
begin
set @x = 15;
set @y = 10;
select @x, @y, @x-@y;
end //
delimiter ;
```

Execute the procedure: call my_procedure_user_variables();

- **MySQL simple IF-THEN statement**

We have to specify a condition for executing the code. If the statement evaluates to true, it will execute the statement between IF-THEN and END-IF. Otherwise, it will execute the statement following the END-IF.

Syntax:
```
IF condition THEN
    statements;
  END IF;
```

Example:
```
delimiter //
create procedure proc_calculate_discount(price numeric (6,2),out discount numeric (6,2))
begin
if (price>200) then
 set discount=price*.5;
end if;
 select discount;
end//
delimiter;
```

Execute:
```
call proc_calculate_discount(600,@d);
+---------------+
| discount |
+---------------+
|      300.00 |
```

- **IF-THEN-ELSE Statement**

If the statement evaluates to true, it will execute the statement between IF-THEN and ELSE. Otherwise, it will execute the statement following the ELSE and END-IF.

Syntax:
```
IF condition THEN
        statements;
ELSE
        else-statements;
END IF;
```

Example:
```
delimiter //
create procedure proc_calculate_price(price numeric(6,2),out discount numeric(6,2))
```

```
begin
if (price>200) then
 set discount=price*.5;
else
set discount=price;
end if;
 select discount;
end//
delimiter;
```

Execute: call proc_calculate_price(600,@d);

➢ IF-THEN-ELSEIF-ELSE Statement

If the condition becomes true, it will execute the IF-THEN branch. Otherwise, it will evaluate elseif-condition. When the elseif-condition becomes true, it will execute the elseif-statement. If this condition is also false, it will evaluate the next elseif-condition. Thus, here we will evaluate multiple elseif-condition, and if any condition in the IF and ELSE-IF does not becomes true, it will execute the statement of the ELSE branch.

Syntax:

```
IF condition THEN
        statements;
ELSEIF elseif-condition THEN
        elseif-statements;
        ...
ELSE
        else-statements;
END IF;
```

Example:

```
delimiter //
create procedure proc_price_discount (price numeric(6,2),out discount numeric(6,2))
begin
if (price>500) then
  set discount=price*.5;
elseif (price<=500 and price>250) then
  set discount=price*.8;
else
  set discount=price;
end if;
select discount;
end //
delimiter ;
```

Execute: call proc_price_discount(350,@c);

➢ MySQL LOOP statement

The **MySQL LOOP** statement could be used to run a block of code or set of statements, again and again, depends on the condition. LEAVE statement is used to exit the flow control construct that has the given label.

Syntax:

```
[begin_label:] LOOP
statement_list
END LOOP [end_label]
```

Example:

```
delimiter //
create procedure proc_loop()
begin
declare x  int;
declare str  varchar(25);
set x = 1;
```

```
                set str = ' ';
        loop_label:  loop
                if  x > 10 then
                        leave  loop_label;
                end  if;
        set  x = x + 1;
                if  (x mod 2) then
                        iterate  loop_label;
                else
                        set  str = concat(str, x,',');
                end  if;
        end loop;
        select str;
end //
delimiter ;
```

Execute: call proc_loop();

```
+-------------+
| str         |
+-------------+
| 2,4,6,8,10, |
```

Example 2: Insert record into table using loop statement

```
create table new_table (value varchar(50) null default null);

delimiter //
create procedure proc_add()
 begin
  declare a int default 1 ;
  simple_loop: loop
    insert into new_table (value) values(a);
    set a=a+1;
    if a=11 then
      leave simple_loop;
    end if;
 end loop simple_loop;
end //
delimiter ;
```

Execute: call proc_add();

```
Select * from new_table;
```

➢ MySQL: RETURN Statement

The RETURN statement terminates execution of a stored function and returns the value *expr* to the function caller. There must be at least one RETURN statement in a stored function. There may be more than one if the function has multiple exit points.

Syntax: RETURN expr

➢ MySQL: REPEAT Statement

The REPEAT statement executes the statement(s) repeatedly as long as the condition is true. The condition is checked every time at the end of the statements.

➢ MySQL: WHILE Statement

The WHILE statement executes the statements as long as the condition is true. The condition is checked every time at the beginning of the loop. Each statement is terminated by a semicolon (;).

Syntax: [begin_label:] WHILE search_condition DO
statement_list

```
        END WHILE [end_label]
```

Example:
```
delimiter //
create procedure proc_while(in n int)
begin
set @sum = 0;
set @x = 1;
while @x<n
do
 if mod(@x, 2) <> 0 then
set @sum = @sum + @x;
end if;
set @x = @x + 1;
end while;
end //
delimiter ;
```

Execute:
```
call proc_while(5);
Select @sum;
```

Stored Procedure

PL/SQL has two types of subprograms:

1. Procedures
2. Functions

A procedure (stored procedure) collection of pre-compiled SQL statements stored inside the database that performs one or more actions. It also known as a subprogram to perform a set of tasks. You can pass information to a procedure using IN, OUT and INOUT parameters. Procedures and Functions are the subprograms which can be created and saved in the database as database objects. A procedure should not return any value to calling area or object.

Features:

- Stored Procedure increases the performance of the applications.
- Once stored procedures are created, they are stored as precompiled object in the database.
- It will be complied once and executed any number of times.
- Stored procedure reduces the traffic between application and database server. Because the application has to send only the stored procedure's name and parameters instead of sending multiple SQL statements.
- Stored procedures are reusable and transparent to any applications.
- A procedure is always secure. The database administrator can grant permissions to applications that access stored procedures in the database without giving any permission on the database tables.
- Procedure can be easily enhansible for future requirement.
- Procedures are explicitly executed by user.

Every procedure having two parts:

- ***Procedure Specification:*** It contains the name of the procedure and the parameters or variables passed to the procedure.
- ***Procedure Body:*** It contains a declaration section, execution section and exception section similar to a general PL/SQL block. Here we are solving actual task.

Syntax:
```
delimiter //
Create [or replace] Procedure <procedure_name> (parameter_name [mode] data type)
                                                (Formal Parameters)
Begin
        <Declaration Statements>;
        <Executable statements>;
        <Data Processing Statements>;   ---SQL Queries
End //
```

How to execute a procedure

```
call <procedure_name> (actual parameter)//
```

Example: Write a pl/sql stored procedure display employee details.

```
delimiter //
create procedure proc_empinfo(veno int)
begin
select * from emp where eno=veno;
end //
```

Execute the procedure:

```
call proc_empinfo (101) //
```

➢ **It shows all stored procedures information.**

Lists all stored procedures in the 'bcs' database

```
show procedure status where db ='bcs';
```

Types of Procedures:

1) Static Procedure

It will not contain any arguments (parameters) variable and it won't take any values from the user. Always display same output.

```
delimiter //
create procedure proc_ename ()
begin
declare  vename varchar(20);
select ename into vename from emp where eno=101;
select vename;
end //
```

2) Dynamic Procedure

A procedure with parameters is known as dynamic procedure. It will be take runtime values from the user and display different output values from different user.

Example: Write a pl/sql stored procedure for passing empno as a parameter display name of employee.

```
delimiter //
create procedure proc_empinfo (veno int(15))
 begin
  declare vename varchar(15);
  select ename into vename from emp where eno=veno;
  select veno,vename;
 end//
```

Execute:

```
call proc_empinfo (102)
```

Parameter used in Procedures:

Parameters are used to pass the values to procedure and also return values from the procedure. We must use two types of parameters.

1) Formal Parameter
2) Actual Parameter

1) **Formal Parameter:** Formal parameters are defined in procedure specification. In formal parameter we are defining parameter name and mode of the parameter.

The parameters make the stored procedure more useful and reusable. A parameter in a stored procedure has one of three modes:

- IN Mode
- OUT Mode

- INOUT Mode

Syntax: [IN | OUT | INOUT] parameter_name datatype (length)

- **IN Mode :**
  - By default procedure parameter having IN Mode. It takes input values at runtime.
  - IN Mode used to pass the values into procedure body.
  - This Mode acts like a 'constant' in procedure body.
  - It is a read-only variable inside the subprograms; their values cannot be changed inside the subprogram.

Example: Write a pl/sql stored procedure to insert a record into dept table using IN parameter.

```
delimiter //
create procedure proc_insert_dept (IN dno int, IN dname varchar(40), loc_id int)
  begin
  insert into dept (dno,dname,loc_id) values(dno,dname,loc_id);
  end //
Delimiter ;
Execute:        call proc_insert_dept(60,'HR',1100);
                Select * from dept;              // verify record is inserted or not.
```

- **OUT Mode :**
  - You want return any values from procedure body then use out parameter.
  - OUT Mode acts like an uninitialized variable in procedure body.
  - We must specify OUT keyword explicitly.
  - The value of an OUT parameter can be changed inside the stored procedure and its new value is return to the calling program.

```
Example 1: delimiter //
                create  procedure proc_out_para( IN a int,  OUT b int)
        begin
                set b=a*a;
                select b;
                end //
            delimiter ;
Execute: call procedure proc_out_para (4,@b);
                +------+
                | b    |
                +------+
                |    4 |
Example 2: retrieves maximum salary using out parameter.
                create procedure my_proc_out (out highest_salary int)
                begin
                select max(sal) into highest_salary from emp;
                end//
                delimiter ;
Execute:  call my_proc_out(@m);
            Select @m;
        +-------+
        | @m    |
        +-------+
        | 28000 |
        +-------+
```

- **IN OUT Mode :**
  - An INOUT parameter is a combination of IN and OUT parameters.
  - Calling program may pass the argument and return output values to the calling program.

- You can change values within procedure. It accepts runtime input values.
- It internally acts like a constant, initialized variable.
- We must specify IN OUT keyword explicitly.
- It is a read-write variable inside the subprograms, their values can be changed inside the subprograms.

Example: Write a pl/sql stored procedure to display count of employee given job using IN OUT parameter.

```
create procedure my_proc_inout (inout countjob int, in vjob char(10))
begin
select count(eno) into countjob from emp where job= vjob;
end //
delimiter ;
```

Execute:

```
call my_proc_inout(@c,'Salesman');
select @c;
+-------+
| @c    |
+-------+
|     3 |
```

Dropping Procedure

Syntax: Drop Procedure <procedure_name>;

Example: Drop Procedure pro1;

Conditional Statement using Procedure

Example: Write a pl/sql stored procedure for passing empno as a parameter display employee number, salary, bonus and his final salary.

```
Delimiter //
create procedure proc_salbonus(veno int(10))
begin
declare vsal int;
declare b int;
declare fsal int;
select sal into vsal from emp where eno=veno;
if (vsal>=5000) then
set b=0.20*vsal;
else
set b=0.10*vsal;
end if;
set fsal=b+vsal;
select veno,vsal,b,fsal;
end //
delimiter ;
```

Function

Function is a named pl/sql block which is used to solve some particular task. We are not allowed to use DML statements in functions.

- Function is a PL/SQL object or subprogram.
- Function also like procedure.
- Function is also known as precompiled objects.
- By default functions return single value.
- Functions are executed only implicitly. Don't uses execute command, just call function.
- Function may or may not take parameters.

Every procedure having two parts:

- ***Function Specification:*** It contains the name of the function and type of the parameters.

- ***Function Body:*** It contains a declaration section, execution section and exception section similar to a general PL/SQL block. Here we are solving actual task.

Syntax:
```
create function <fun_name> ( argument1, argument2.........)
return <datatype>
begin
<assigenment stmt>
<data processing stmt>
<output stmt>
return (value/varibale value);
end  //
```

How to call function

Select fun_name (argument value...);

Example: A function taking 2 arguments and returning multiplication of these value.
```
create or replace function fun_multi(x int, y int)
returns int
is
declare z int;
begin
set z=x*y;
return z;
end //
```

1) Calling a function from select query
```
select fun_multi(10,20) from dual;
```

2) Calling a function from procedure
```
create procedure proc_fun_multi ( x1 int, x2 int, x3 OUT int)
begin
set x3=fun_multi(x1,x2);
end //
```

Example: Write a pl/sql function for passing emp no as parameter return gross salary from emp table bases on following condition.
Gross:= basic salary + hra + da-pf;
hra=10% of salary, da=20% of salary, pf=10% of salary
```
create function fun_gross_sal (veno int)
begin
returns  int
declare vsal int;
declare gross int;
declare hra int;
declare da int;
declare pf int;
select sal into vsal from emp where eno=veno;
set hra=vsal*0.1;
set da=vsal*0.2;
set pf=vsal*0.1;
set gross= vsal + hra + da-pf;
return gross;
end //
Execution: select fun_gross_sal(103) from dual;
```

Example: Function to calculate bonus
```
create or replace function f_bonus( s int)
```

```
returns int
begin
declare b int;
if(s>=1000) then
set b=0.05*s;
elsif(s>=2000 and s<3000) then
set b=0.10*s;
elsif (s>=3000 and s<5000) then
set b=0.15*s;
else
set b=0.25*s;
end if;
return(b);
end //
```

Procedure 1:Display eno,ename,sal,bounus and final salary of given eno?

```
create procedure proc_empfsal(veno int)
begin
declare vsal int;
declare bonus int;
declare fsal int;
select sal into vsal from emp where eno=veno;
set bonus=f_bonus(vsal); -- function calling statement
set fsal=bonus+vsal;
select veno, vsal,bonus,fsal;
end //
```

Similarities between Procedure and Function

- Both can be called from other PL/SQL blocks.
- If the exception raised in the subprogram is not handled in the subprogram exception handling section, then it will propagate to the calling block.
- Both can have as many parameters as required.
- Both are treated as database objects in PL/SQL.

Difference between Procedure and Function

Procedure	Function
1. Procedure doesn't contain return clause in header & executable part.	1. Function contain return clause in header & executable part.
2. Procedure may or may not be returning a value.	2. Function always returns a value.
3. Procedure can't be called into select statements.	3. Function can be called into select statement.
4. Procedure is used to implement business logic.	4. Function is used for numeric calculation.

Cursor

Cursor is a temporary memory area (context area) where executes SQL statements. MySQL associates every SELECT statement with a cursor to hold the query information in this context area. A cursor is responsible to repeat a set of rows returned by a select query statement where each row is processed one by one. Thus, Cursor is a reserved SQL area from where information in a database can be retrieved. The cursor is convenient to use when you are performing on a complex result-set. The cursor can be used inside the stored procedures, functions and triggers.

MySQL cursor has three types of properties.

1. Read Only

You cannot update data in the underlying table through the cursor.

2. Non_Scrollable

Only rows can be retrieved in the order specified by the SELECT statement. In the reverse order,

users can not retrieve records. Additionally, in the result set, users cannot skip rows or jump to a particular row.

3. Asensitive

An asensitive cursor is used to points the actual data, whereas a temporary copy of the data is used by an insensitive cursor used. An asensitive cursor performs faster than an insensitive cursor because it does not have to make a temporary copy of data.

Working with MySQL Cursors

We have to following steps while using the MySQL Cursors:

- Declare a Cursor
- Open a Cursor
- Fetch the Cursor
- Close the Cursor
- Handler

1. Declaration of a Cursor

The cursor declaration must be after any variable declaration. If you declare a cursor before the variable declarations, MySQL will issue an error. A cursor must always associate with a SELECT statement.

```
Syntax: DECLARE <cursor_name> CURSOR FOR <select_statement>;
```

2. Open a Cursor

OPEN statement used before fetching rows from the result set.

```
Syntax: OPEN <cursor_name>;
```

3. Fetch the Cursor

Use the FETCH statement to retrieve the next row pointed by the cursor and move the cursor to the next row in the result set.

```
Syntax: FETCH <cursor_name> INTO <variable_list>;
```

4. Close the Cursor

Deactivate the cursor and release the memory associated with it using the CLOSE statement. We can close the previously opened cursor.

```
Syntax: CLOSE <cursor_name>;
```

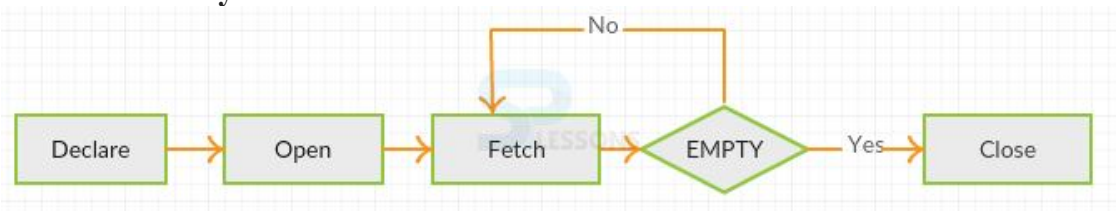
5. To declare a NOT FOUND handler:

When working with MySQL cursor, you must also declare a NOT FOUND handler to handle the situation when the cursor could not find any row.

FETCH statement, the cursor attempts to read the next row in the result set. Where finished is a variable to indicate that the cursor has reached the end of the result set.

```
Syntax: declare continue handler for not found set finished = 1;
```

MySQL Cursor Life Cycle



Examples:

1. Display all employee whose job is 'CLERK'.

```
delimiter //
create  procedure empjobs()
begin
   declare veno int;
   declare vename varchar(10);
   declare vsal int;
```

```
  declare done int default false;
  declare cur_empjobs cursor for select eno,ename,sal from emp where job='CLERK';
  declare continue handler for not found set done = true;
  open cur_empjobs;
  read_loop:loop
  fetch cur_empjobs into veno,vename,vsal;
   if done then
      leave read_loop;
  end if;
select veno,vename,vsal;
   end loop;
   close cur_empjobs;
   end //
```

2. Find the median of the salary paid to all employees

```
Delimiter //
create procedure median ()
  begin
   declare  nemps, n    integer;
   declare  mediansal   float;
    declare  sal_cursor  cursor for select sal from emp order by sal;
  select count(*)  into nemps from emp;
    open sal_cursor;
   set n = 0;
   l: loop
       fetch sal_cursor into mediansal;
            if n < nemps /2 then
               set n = n + 1;
               iterate  l;
            end if;
        select mediansal;
            leave l;
      end loop ;
   close sal_cursor;
  end //
  delimiter ;
```

3. Find the average of the salary all employees

```
delimiter //
create procedure procavgsal()
begin
declare asal int;
declare done int default false;
declare c1 cursor for select avg(sal) from emp;
declare continue handler for not found set done=true;
open c1;
read_loop:loop
fetch c1 into asal;
if done then
leave read_loop;
end if;
select asal;
end loop;
```

```
close c1;
end  //
```

4. Inserting Record into another table through cursor.

```
Create table emp1( eno int primary key, sal int);

delimiter //
create procedure sal_cur2()
begin
 declare veno int;
 declare vsal int;
declare done int default false;
 declare c2 cursor for select eno,sal from emp;
 declare continue handler for not found set done=true;
open c2;
emp_loop:loop
 fetch c2 into veno,vsal;
 insert into emp1(eno,sal) values (veno,vsal);
 if done then
 leave emp_loop;
 end if;
 end loop;
 close c2;
end  //
```

5. WAB which accept department no from user and increment the salary of all the employees working in that dept by 5%.

```
delimiter //
create procedure update_sal(vdno int)
begin
declare veno,vsal int;
declare vename varchar(15);
declare done int default false;
declare c_emp cursor for select eno,ename,sal from emp where dno=vdno;
declare continue handler for not found set done=true;
open c_emp;
upsal:loop
fetch c_emp into veno,vename,vsal;
if done then
leave upsal;
end if;
update emp set sal=vsal+vsal*0.05;
select veno,vename,vsal;
select eno,ename,sal from emp where dno=vdno;
end loop;
close c_emp;
end //
```

- **Advantages:**
  1. Cursors are best when you want fetch each row one by one.
  2. It is efficient because there is no need to write complex queries(like joins)
- **Disadvantages:**
  1. Cursor is faster than a while looping but it creates more overhead in database.
  2. Cursor fetching one by one data from database so if data is more, its take more execution time.

Triggers

A trigger in MySQL is a set of SQL statements that reside in a system catalog. Trigger is a database object. It will be permantly saved in database. Triggers are same as stored procedures that are automatically run in response to an event. The event could be an insert, an update, or a delete statement performed against table or view. These procedures are called database triggers. Trigger is executed implicitly whenever the triggering event occurs.

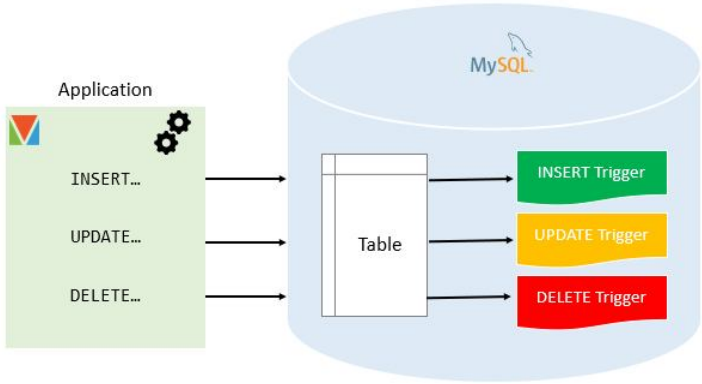
Trigger is invoked before the data entered as record in the table, this data taken by trigger. This is responsibility of trigger.

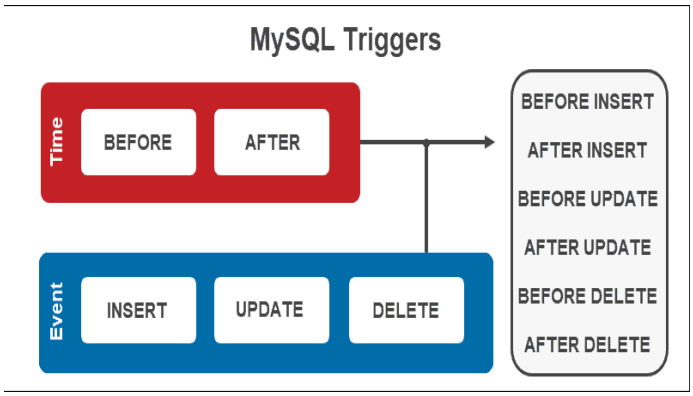
Need of Triggers

1) To maintain business data into uniformly case because submitted data from the user in any case upper or lower.
2) To maintain audit information of any table data. Audit information means monitoring table data what will happen on data.
3) Triggers help us to validate data even before they are inserted or updated.
4) Triggers increase the performance of SQL queries because it does not need to compile each time the query is executed.
5) Triggers reduce the client-side code that saves time and effort.
6) Triggers help us to scale our application across different platforms.

Advantages of triggers :

- Audit data modifications
- Log events transparently
- Enforce complex business rules
- Derive column values automatically
- Implement complex security authorizations
- Maintain replicate tables
- Faster data updates. We do not need to wait for scheduled tasks and scripts to run it update data.
- Maintain the integrity of data
- Better error-handling capacity





Triggers are two types

- **Row-Level Trigger:** It is a trigger, which is activated for each row by a triggering statement such as insert, update, or delete. Example: if a table has inserted, updated, or deleted multiple rows, the row trigger is fired automatically for each row affected by the insert, update, or delete statement.
- **Statement-Level Trigger:** It is a trigger, which is fired once for each event that occurs on a table regardless of how many rows are inserted, updated, or deleted.

Trigger Classification

Triggers can be classified based on the following parameters.

Classification based on the timing

- **Before Trigger:** It fires before the specified event has occurred.
- **After Trigger:** It fires after the specified event has occurred.

Classification based on the level

- **Statement Level Trigger:** It fires one time for the specified event statement.
- **Row Level Trigger:** It fires for each record that got affected in the specified event. (only for DML)

Classification based on the Event

- **DML Trigger:** It fires when the DML event is specified (Insert/Update/Delete)
- **DDL Trigger:** It fires when the DDL event is specified (Create/Alter)
- **DATABASE Trigger:** It fires when the database event is specified. (Logon / Logoff / Startup / Shutdown)

So each trigger is the combination of above parameters.

➢ **A trigger has three basic parts:-**

- trigger event or statement
- trigger restriction
- trigger action

- **Trigger event or statement:** A trigger event can be an insert, update or delete statement for a specific table.
- **Trigger Restriction:** It controls the trigger executions before or after event.
- **Trigger Action:** it implements or represent the logic or functionality of trigger. i.e. what trigger doing.

Combination trigger are combination of two trigger type:

- **Before Statement Trigger:** fire only once for each statement before the triggering DML statement.
- **Before Row trigger:** fire for each and every record before the triggering DML statement.
- **After Statement Trigger:** fire only once for each statement after the triggering DML statement executing.
- **After Row Trigger:** fire for each and every record after the triggering DML statement executing.

Keywords:

1) New- it will get new value from column.
New is associated with insert and update operation.
Syntax- new.column_name
Example- new.ename

2) Old - it will get old value from column.
Old is associated with delete and before update operation.
Syntax- old.column_name
Example- old.sal

Trigger Event	Old	New
Insert	No	Yes
Update	Yes	Yes
Delete	Yes	No

```
Syntax: create trigger  <trigger_name>
                { before |after} { insert, update, delete }
        on  table_name   for each row | for each statement [when condition ]
  begin
        variable declarations
        trigger code
   end //
```

Trigger executes BEFORE to convert ename field lowercase to uppercase.

```
delimiter //
create trigger trg1
before insert on emp for each row
begin
  set new.ename=upper(new.ename);
end //
delimiter ;
insert into emp values (116,'nitin','manager','1989-05-23',20000,30);
```

```
select * from emp;
```

Restriction to Deleting Trigger- This trigger is preventing to deleting row.

```
delimiter //
create trigger trg2
        after delete on emp for each row
begin
        if old.eno=101 then
signal sqlstate '50002' set message_text = 'you can't delete this row';
        end if;
end//
delete from emp where eno = 101;
```

--creates a log table and a trigger that inserts a row in the log table after any UPDATE statement affects the SALARY column of the EMPLOYEES table, and then updates EMPLOYEES.SALARY and shows the log table.
Trigger Logs Changes to EMPLOYEES.SALARY
Create log table:

```
drop table emp_log;
create table emp_log (
  eno int,
  log_date date,
  salary int,
  vuser varchar(20));
```

Create trigger that inserts row in log table after employee salary is updated:

```
delimiter //
create trigger log_salary_increase
  after update on emp for each row
begin
declare vuser varchar(50);
select user() into vuser;
 insert into emp_log values (new.eno,sysdate(),old.sal,vuser);
end//

Update Employees Salary:  update emp set sal= sal+1000  where dno=30;

Show log table:  select * from emp_log;
```

Before Insert Trigger:

The before insert trigger gives control over data modification before committing into a database table.

Ex: Write a before insert trigger to check the age value before inserting data into the *person* table

```
create table person (name varchar(45), age int);

delimiter //
create trigger person
before insert on person for each row
if new.age < 18 then
signal sqlstate '50001' set message_text = 'person must be older than 18.';
end if; //
delimiter ;
```

```
mysql> INSERT INTO person VALUES ('John', 14);
ERROR 1644 (50001): Person must be older than 18.
```

After Insert Trigger

The AFTER INSERT trigger is useful when the entered row generates a value needed to update another table.

Create a table called average_age with a column called average:

```
create table average_age (average  double);
```

Insert the average age value into the table:

```
insert into average_age select avg(age) from person;
```

Ex: Create an AFTER INSERT trigger on the person table to update the average_age table after insert:

```
delimiter //
create trigger person_ai
after insert on person for each row
update average_age set average = (select avg(age) from person); //
delimiter ;
```

```
mysql> INSERT INTO person VALUES ('John', 19);
Query OK, 1 row affected (0.00 sec)

mysql> SELECT * FROM person;
+---------+------+
| name    | age  |
+---------+------+
| Matthew |   25 |
| Mark    |   20 |
| John    |   19 |
+---------+------+
3 rows in set (0.00 sec)

mysql> SELECT * FROM average_age;
+--------------+
| average      |
+--------------+
| 21.333333333 |
+--------------+
1 row in set (0.00 sec)
```

➢ **Show Triggers**

List all the triggers in a database with:

```
show triggers;
```

➢ **Delete Triggers**

To delete a trigger, use the DROP TRIGGER statement:

```
drop trigger <trigger name>;
drop trigger stud;
```

📣 **Transactions**

- A Transaction in SQL is a single logical unit of work.
- Transactions can make multiple changes to the database like create tables, delete records, update records, etc in the same database transaction.
- The transaction is successful when all the operations in it complete and are committed which means that the changes are committed to memory i.e. made permanent.
- A transaction fails if any of the operations in it fail and cause a rollback. A rollback means that the changes are discarded and are not permanent.
- Commit or Rollback is either implicit or explicit.
- In MySQL, transactions with DDL instructions like CREATE, ALTER, RENAME, DROP and TRUNCATE are implicitly committed. The changes once made are permanent and users cannot control this.
- Users can switch off auto-commit for DML instructions like INSERT, UPDATE, DELETE and SELECT.
- Transactions follow the ACID properties.

MySQL provides us with the following important statement to control transactions:

- To start a transaction, you use the START TRANSACTION statement. The BEGIN or BEGIN

WORK are the aliases of the START TRANSACTION.

- To commit the current transaction and make its changes permanent, you use the COMMIT statement.
- To roll back the current transaction and cancel its changes, you use the ROLLBACK statement.
- To disable or enable the auto-commit mode for the current transaction, you use the SET auto commit statement.

By default, MySQL automatically commits the changes permanently to the database. To force MySQL not to commit changes automatically, you use the following statement:

```
set autocommit = 0;   or   set autocommit = off
```

You use the following statement to enable the auto commit mode explicitly:

```
set autocommit = 1;   or   set autocommit = on;
```

```
Syntax:  START TRANSACTION
         [transaction_characteristic [, transaction_characteristic] ...]
         transaction_characteristic:
         {
         WITH CONSISTENT SNAPSHOT
           | READ WRITE
           | READ ONLY
         }
         BEGIN [WORK]
         COMMIT [WORK] [AND [NO] CHAIN] [[NO] RELEASE]
         ROLLBACK [WORK] [AND [NO] CHAIN] [[NO] RELEASE]
         SET autocommit = {0 | 1}
```

These statements provide control over use of transactions :

- The START TRANSACTION or BEGIN statement begins a new transaction.
- COMMIT commits the current transaction, making its changes permanent.
- ROLLBACK rolls back the current transaction, canceling its changes.
- The SET auto commit statement disables or enables the default auto commit mode for the current session.

```
-- 1. Start a new transaction
 start transaction;
-- 2. Get the highest income
 select @sal:= max(sal) from emp;
-- 3. Insert a new record into the employee table
 insert into emp values (116,'nitin','manager','1989-05-23',20000,30);
-- 5. Commit changes
commit;
```

Rollback:

```
start transaction;                    -- 1. Start a new transaction
 delete from emp;                     -- 2. Delete data from the order table
-- After the execution of the above statement then execute
select * from emp;
rollback;                   -- 3. Rollback changes
select * from emp;          -- 4. Verify the records in the first session
```

Savepoint, Rollback To Savepoint, Release Savepoint:

```
start transaction;
select * from emp;
insert into emp values (117,'prakash','admin','1990-06-23',18000,10);
savepoint my_savepoint;
```

```
insert into emp values (118,'ganesh','clerk','1988-10-12',16000,10);
rollback to savepoint my_savepoint;
insert into emp values (119,'santosh','salesman','1991-10-26',14000,30);
commit;
```

The **RELEASE SAVEPOINT** statement destroys the named savepoint from the current transaction without undoing the effects of queries executed after the savepoint was established. After these statements, no rollback command occurs. If the savepoint does not exist in the transaction, it gives an error.

```
start transaction;
insert into emp values (120,'salman','salesman','1993-12-09',14000,30);
savepoint my_savepoint;
update emp set ename='amir' where eno=118;
release savepoint my_savepoint;
commit;
```

➢ MySQL Table Locking

A lock is a mechanism associated with a table used to restrict the unauthorized access of the data in a table. **MySQL allows a client session to acquire a table lock explicitly to cooperate with other sessions to access the table's data**. MySQL also allows table locking to prevent it from unauthorized modification into the same table during a specific period.

A session in MySQL can acquire or release locks on the table only for itself. Therefore, one session cannot acquire or release table locks for other sessions. It is to note that we must have a TABLE LOCK and SELECT privileges for table locking.

Table Locking in MySQL is mainly **used to solve concurrency problems**. It will be used while running a transaction, i.e., first read a value from a table (database) and then write it into the table (database).

MySQL provides **two types of locks** onto the table, which are:

READ LOCK: This lock allows a user to only read the data from a table.

WRITE LOCK: This lock allows a user to do both reading and writing into a table.

```
Syntax: lock tables table_name [read | write];
Ex:  lock table emp read;
```

Once the READ lock is acquired on to the table, we cannot write data to the table in the same session.

Unlock Tables Statement

It allows us **to release a lock** for a table.

```
unlock tables;
```

📢 Import CSV File Into MySQL Table

The LOAD DATA INFILE statement to import CSV file into MySQL table. This statement allows you to read data from a text file and import the file's data into a database table very fast.

Before importing the file, you need to prepare the following:

- A database table to which the data from the file will be imported.
- A CSV file with data that matches with the number of columns of the table and the type of data in each column.
- The account, which connects to the MySQL database server, has FILE and INSERT privileges.

CREATE TABLE statement to create the discounts table

```
create table discounts (
  id int not null auto_increment,
  title varchar(255) not null,
  expired_date date not null,
```

```
amount decimal(10 , 2 ) null,
primary key (id));
```

The following discounts.csv file contains the first line as column headings and other three lines of data.

```
discounts.csv
1 id,title,expired date,amount
2 1,"Spring Break 2014",20140401,20
3 2,"Back to School 2014",20140901,25
4 3,"Summer 2014",20140825,10
```

Imports data from the c:\tmp\discounts.csv file into the discounts table.

```
load data infile 'c:/tmp/discounts.csv'
into table discounts
fields terminated by ','
enclosed by ' " '
lines terminated by '\n'
ignore 1 rows;
```

The field of the file is terminated by a comma indicated by FIELD TERMINATED BY ',' and enclosed by double quotation marks specified by ENCLOSED BY ' " '.
Each line of the CSV file is terminated by a newline character indicated by LINES TERMINATED BY '\n' .
Because the file has the first line that contains the column headings, which should not be imported into the table, therefore we ignore it by specifying IGNORE 1 ROWS option.
Now, we can check the discounts table to see whether the data is imported.

```
select * from discounts;
```

MySQL Export Table to CSV

MySQL provides an easy way to export the query's result into a CSV file that resides in the database server.Before exporting data, you must ensure that:

- The MySQL server's process has the write access to the target folder that contains the target CSV file.
- The target CSV file must not exist.

To export this result set into a CSV file,

```
select orderno, status, orderdate, requireddate, comments from orders where status = 'cancelled'
into outfile 'c:/tmp/cancelled_orders.csv'
fields enclosed by ' " '
terminated by ';'
escaped by ' " '
lines terminated by '\r\n';
```

The statement created a CSV file named cancelled_orders.csv in the C:\tmp folder that contains the result set.